第一题

本人一直大一采用 VMWare workstation 上的 ubuntu 系统，在 VS Code 配置好的 cpp 环境下，使用 gcc/g++编译链接并构建可执行文件，采用 gdb 进行调试。

第二题

代码：

```
#include <iostream>
#include <vector>
using namespace std;
/**
 * 维护堆的性质
 * @param arr 存储堆的数组
 * @param n 数组长度
 * @param i 待维护节点的下标
*/
void heapify(vector<int>& arr, int n, int i)
{
	int largest = i;
	int lson = i*2+1;
	int rson = i*2+2;

	if(lson < n && arr[largest]<arr[lson])
		largest = lson;
	if(rson < n && arr[largest]<arr[rson])
		largest = rson;
	if(largest != i)
	{
		swap(arr[largest],arr[i]);
		heapify(arr, n, largest);
	}
}
// 堆排序入口
void heap_sort(vector<int>& arr)
{
	int i,n=arr.size();
	// 建堆
	for(i=n/2-1; i >= 0;i--)
		heapify(arr, n, i);
	// 排序
	for (i=n-1; i>0; i--)
	{
		swap(arr[i], arr[0]);
		heapify(arr,i ,0);
```

```
        }

    }
    // 打印数组
    void printArray(const vector<int>& arr) {
        for (int val : arr)
            cout << val << " ";
        cout << endl;
    }
    int main() {
        // 输入数组
        vector<int> arr;
        int n, temp;
        cout << "请输入数组的大小: ";
        cin >> n;
        cout << "请输入数组的元素: ";
        for (int i = 0; i < n; ++i) {
            cin >> temp;
            arr.push_back(temp);
        }
        // 调用堆排序函数
        heap_sort(arr);
        // 输出排序后的数组
        cout << "排序后的数组: ";
        printArray(arr);
        return 0;
    }
```

运行结果：

```
请输入数组的大小： 10
请输入数组的元素： 1 5 78 12 4 5 6 12 45 6
排序后的数组： 1 4 5 5 6 6 12 12 45 78
```

第三题

问题分析解答：

定义了 processNumber(int number, function<int(int)> callback) 函数，它接收一个整数和一个回调函数，以及加法函数 add(int x, int increment = 1)、平方函数 square(int x)、乘法函数 int multiply(int x, int factor = 2)。在 main()中调用 processNumber 函数时，由于接受的函数参数只能为单个参数的函数，故加法函数 add(int x, int increment = 1)和乘法函数 int multiply(int x, int factor = 2)是通过 lambda 表达式创造新的只接受一个参数的函数调用。

代码：

```
#include <iostream>
#include <functional>
using namespace std;
// 定义 processNumber 函数，它接收一个整数和一个回调函数
```

```
void processNumber(int number, function<int(int)> callback) {
    int result = callback(number);
    cout << "处理后的结果: " << result << endl;
}
// 定义加法函数
int add(int x, int increment = 1) {
    return x + increment;
}
// 定义平方函数
int square(int x) {
    return x * x;
}
// 定义乘法函数
int multiply(int x, int factor = 2) {
    return x * factor;
}
int main() {
    int number;
    cout << "请输入一个整数: ";
    cin >> number;

    cout << "增加 5,";
    processNumber(number, [](int x) { return add(x, 5); });  // 增加 5
    cout << "平方运算， ";
    processNumber(number, square);  // 平方
    cout << "乘以 3,";
    processNumber(number, [](int x) { return multiply(x, 3); });  // 乘以 3

    return 0;
}
```

运行结果：

```
请输入一个整数： 5
增加5,处理后的结果： 10
平方运算， 处理后的结果： 25
乘以3,处理后的结果： 15
```

第四题

问题解答：

1. 程序不会报错，但超出数组边界的值会输出错误的结果；
2. sizeof 是一个操作符，不是函数。它用于获取对象或类型所占的内存大小（以字节为单位）；
3. 在 main 函数中，arr 是一个数组，因此 sizeof(arr) 会返回整个数组所占的内存大小。对于 int arr[5]， int 类型占 4 字节，则 sizeof(arr) 将返回 20（5 * 4 = 20 字节）。然而，在 traversalArray 函数中，arr 被声明为函数参数，此时它被视为指向数组第一个元素的指针。因此，sizeof(arr) 在这里将返回指针的大小，而不是数组的大小。32 位系统上是 4 字节，

64 位系统上是 8 字节。我的操作系统是 64 位输出 8。

4. 数组衰减是指在函数参数传递过程中，数组名会被转换为指向数组第一个元素的指针。这意味着，在函数内部，我们无法通过数组名来获取数组的实际大小（即元素个数），因为此时它已经被转换为一个指针。因此，以 traversalArray 函数为代表的数组遍历函数需要数组的长度作为额外参数。这是因为函数内部无法直接通过 sizeof(arr) 来获取数组的长度；它只能获取到指针的大小。为了正确遍历数组，我们必须从函数外部传递数组的长度信息。

代码：

```
#include <iostream>
using namespace std;

void traversalArray(int arr[], int size) {

	for (int i = 0; i < size; ++i) {
		cout << arr[i] << " ";
	}
	cout << endl;
	cout << "traversalArray 函数中，执行 sizeof(arr):" << sizeof(arr) << endl;
}

int main() {
	int arr[5] = {10, 20, 30, 40, 50};
	cout << "main 函数中，执行 sizeof(arr):" << sizeof(arr) << endl;
	traversalArray(arr, 10); // 如果这里传递 10 作为 size，将会导致越界访问
	return 0;
}
```

运行结果：

```
main函数中， 执行sizeof(arr):20
10 20 30 40 50 0 -1314543360 57461345 1 0
traversalArray函数中， 执行sizeof(arr):8
```

第五题

代码：

```
#include <iostream>
#include <cmath>
using namespace std;

class Complex {
private:
	double x;
	double y;

public:
	Complex(double real, double imag) : x(real), y(imag) {}
```

```
    double getx() const {
        return x;
    }
    double gety() const {
        return y;
    }
    void Display() const {
        if (y >= 0)
            cout << x << "+" << y << "i";
        else
            cout << x << y << "i";
        cout << endl;
    }
    double Abs() const {
        return sqrt(x * x + y * y);
    }
    Complex Minus(const Complex &other) const {
        return Complex(x - other.x, y - other.y);
    }
    Complex Multiply(const Complex &other) const {
        return Complex(x * other.x - y * other.y, x * other.y + y * other.x);
    }
};

int main() {
    Complex z1(1.4, -2.3);
    Complex z2(-3.5, 2.7);
    cout << "z1 复数的模: " << z1.Abs() << endl;
    Complex a = z1.Minus(z2);
    cout << "z1 - z2 : ";
    a.Display();
    Complex b = z1.Multiply(z2);
    cout << "z1 * z2 : ";
    b.Display();
    return 0;
}
```

运行结果：

```
z1复数的模: 2.69258
z1 - z2 : 4.9-5i
z1 * z2 : 1.31+11.83i
```